Class: TY-BTech AIML (SEM VI)

Subject: LLM and Agentic AI

Experiment No.	9
Title	Building an AI Agent: an Autonomous Email Assistant (LLM + Memory + Tools + Orchestration)
Lab Outcome Achieved	Designed and built an autonomous AI email agent that combines a local LLM (Qwen2.5-7B via Ollama) for intent understanding and planning, tool calling for contact lookup and Gmail SMTP delivery, short- and long-term memory for follow-up instructions, and code-enforced guardrails (address verification and human confirmation) around the irreversible send action.
Date of Performance	16/10/2026
Date of submission	16/10/2026
APP ID	2307040
Name of the Student	Anson Dias

Theory:

A chatbot only produces text; an **AI agent** pursues a goal by deciding what to do, doing it through tools, observing the result and continuing until the goal is met. A useful working definition is **AI Agent = LLM (reasoning/planning) + Memory (context/state) + Tools (actions) + Orchestration**. The aim of this experiment is to build such an agent end-to-end: an email assistant that accepts an instruction such as “Send an email to Dr. Meher saying my Experiment 9 agent is complete” and autonomously understands it, looks up the recipient, drafts the email, asks for confirmation, sends it through Gmail and remembers the interaction for follow-ups.

1. The four components of an agent

Component	Role	This implementation
LLM	Understands the request, plans the next step, chooses tools, writes text	Qwen2.5-7B-Instruct (Q4_K_M) served locally by Ollama, temperature 0 (llm.py)
Memory	Keeps context within a conversation and state across sessions	Short-term: last 12 turns replayed to the LLM; long-term: every email with its status in data/memory.json (memory.py)
Tools	Let the agent act on the world and fetch facts it does not know	lookup_contact, save_contact, recall_recent_emails, send_email, resend_email (tools.py)
Orchestration	Runs the loop, passes results back, enforces limits and safety	understand → ReAct tool loop → reply, step limit, stall recovery, guardrails (agent.py)

2. Tool calling and the ReAct loop

Modern LLMs support **function (tool) calling**: each tool is described to the model by a name, a description and a JSON schema of its arguments. Instead of answering in prose the model can emit a structured call such as lookup_contact(name="Dr. Meher"). The orchestrator executes the real function and appends its result (the observation) to the conversation, and the model reasons again with the new information. This Reason → Act → Observe cycle is the **ReAct** pattern. The loop ends when the model replies in plain text, and a hard step limit (8) prevents runaway loops:

```
messages = [system_prompt, *recent_turns, current_task_note]
for step in range(MAX_AGENT_STEPS):
    reply = llm.chat(messages, tools=TOOL_SCHEMAS)
    if not reply.tool_calls:  return reply.content        # final answer
    for call in reply.tool_calls:
        result = toolbox.run(call.name, call.arguments)   # guardrails live here
        messages.append({"role": "tool", "content": result})
```

3. Intent and entity extraction with structured output

Step 1 of the agent turns free text into structured data. Ollama supports **grammar-constrained decoding**: when a JSON schema is passed in the format field, token sampling is restricted so the output is always valid JSON matching the schema. The schema used here extracts intent (send_email, resend_previous, provide_info, query_history, cancel, other), recipients exactly as named, a subject, key points, the tone (formal/friendly/neutral/urgent) and whether the message refers to an earlier email. The parsed request is pinned into the next stage as a “current task” note so the planner acts on this instruction and not on older ones in the history.

4. Memory: short-term and long-term

Short-term memory is the conversation buffer: the most recent turns are replayed in the model’s context window on every request, so a reply like “It’s john.doe@example.com” is understood as the answer to the agent’s question. **Long-term memory** is a persistent store outside the model: every drafted, sent, declined or failed email is written to data/memory.json and read back with the recall_recent_emails tool. A follow-up such as “Send the same email to Sarah too” needs both: the reference “the same email” is resolved to a stored email, and resend_email reuses its subject and body word-for-word (only the greeting changes), so the copy cannot drift from what was originally approved. Because the store is on disk, follow-ups work even after the program is restarted.

5. Guardrails for an irreversible action

LLMs can **hallucinate**: asked to email “John”, a model may invent a plausible address such as john@gmail.com. Sending an email cannot be undone, so the safety rules are enforced in code at the tool boundary rather than trusted to the prompt:

- **Address verification (grounding):** send_email, resend_email and save_contact accept an address only if it is in the contacts store or was literally typed by the user in the conversation. Anything else returns BLOCKED to the model, which must look the person up or ask.
- **Human-in-the-loop confirmation:** the full draft (From, To, Subject, body) is shown and nothing is sent without an explicit “y”. “n” cancels; any other text is returned to the model as revision feedback. Setting CONFIRM_BEFORE_SEND=false switches to direct sending, but verification still applies.
- **Idempotency:** the same email cannot be sent twice for one instruction, and a declined recipient is not retried.

Each guardrail returns a clear observation (BLOCKED, NOT_SENT, ALREADY_SENT) that the model can reason about, so a blocked action leads to recovery (asking the user) rather than a crash.

6. Sending email: SMTP and Gmail App Passwords

Email is delivered with the **Simple Mail Transfer Protocol (SMTP)**. Python’s smtplib opens an implicitly encrypted TLS connection to smtp.gmail.com on port 465, authenticates and submits an RFC 5322 message built with email.message.EmailMessage. Google does not accept the normal account password from programs. Instead, after 2-Step Verification is enabled, the account owner generates a **16-character App Password** for one application. It can be revoked at any time without changing the

main password. Here it is entered once through a hidden prompt (setup_credentials.py), checked against Gmail and stored in a .env file with owner-only permissions. The LLM never sees the credential: it can only ask the send tool to send, and the tool reads the secret from configuration.

7. System design and execution environment

Figure 0 shows the complete agent. Everything except email delivery runs locally on an Apple M1 MacBook (8 GB RAM): the LLM is qwen2.5:7b (4.7 GB, 4-bit) on Ollama, and the agent uses only the Python 3.14 standard library (urllib, smtplib, imaplib, json). No cloud LLM or API key is used.

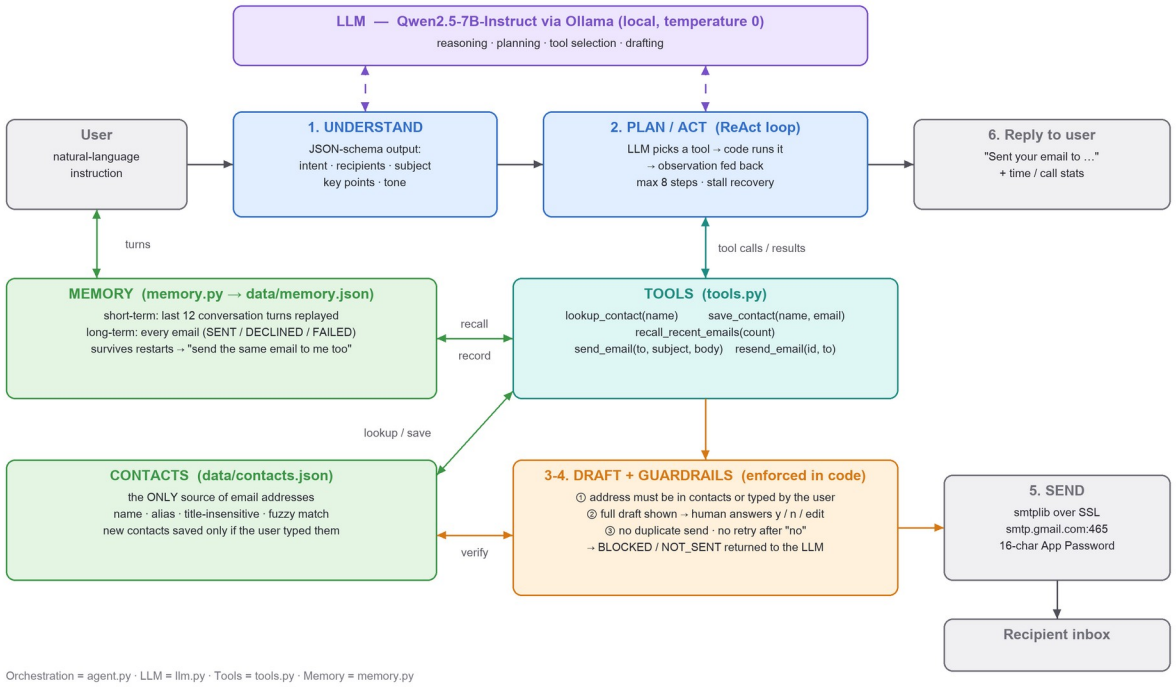


Fig. 0 — Architecture of the AI email agent: LLM, orchestration loop, tools, guardrails, memory and delivery.

Output:

The agent was run live on 16/10/2026 on the Apple M1 MacBook. Each scenario was started as a separate process (python3 run_demo.py N) so that the follow-up steps also prove that memory persists across restarts. The figures show the verbatim terminal output of each run. The human confirmations in the demo runner were scripted, but a “y” was only allowed for a draft whose exact recipient, subject and body I had reviewed and approved beforehand in a dry run (checked by hash). Any other draft would have been answered “n” automatically.

Step 1 — Project, contacts store and configuration

The project has one module per agent component. The contacts store holds the only addresses the agent may use: the instructor and the owner (alias “me”). There is deliberately no entry for “John”, to test the missing-contact path. The model is qwen2.5:7b on Ollama, and the agent runs with CONFIRM_BEFORE_SEND=True. The Gmail App Password was created in the Google account and stored with setup_credentials.py. It sits in a .env file readable only by the owner (-rw-------) and is never printed.

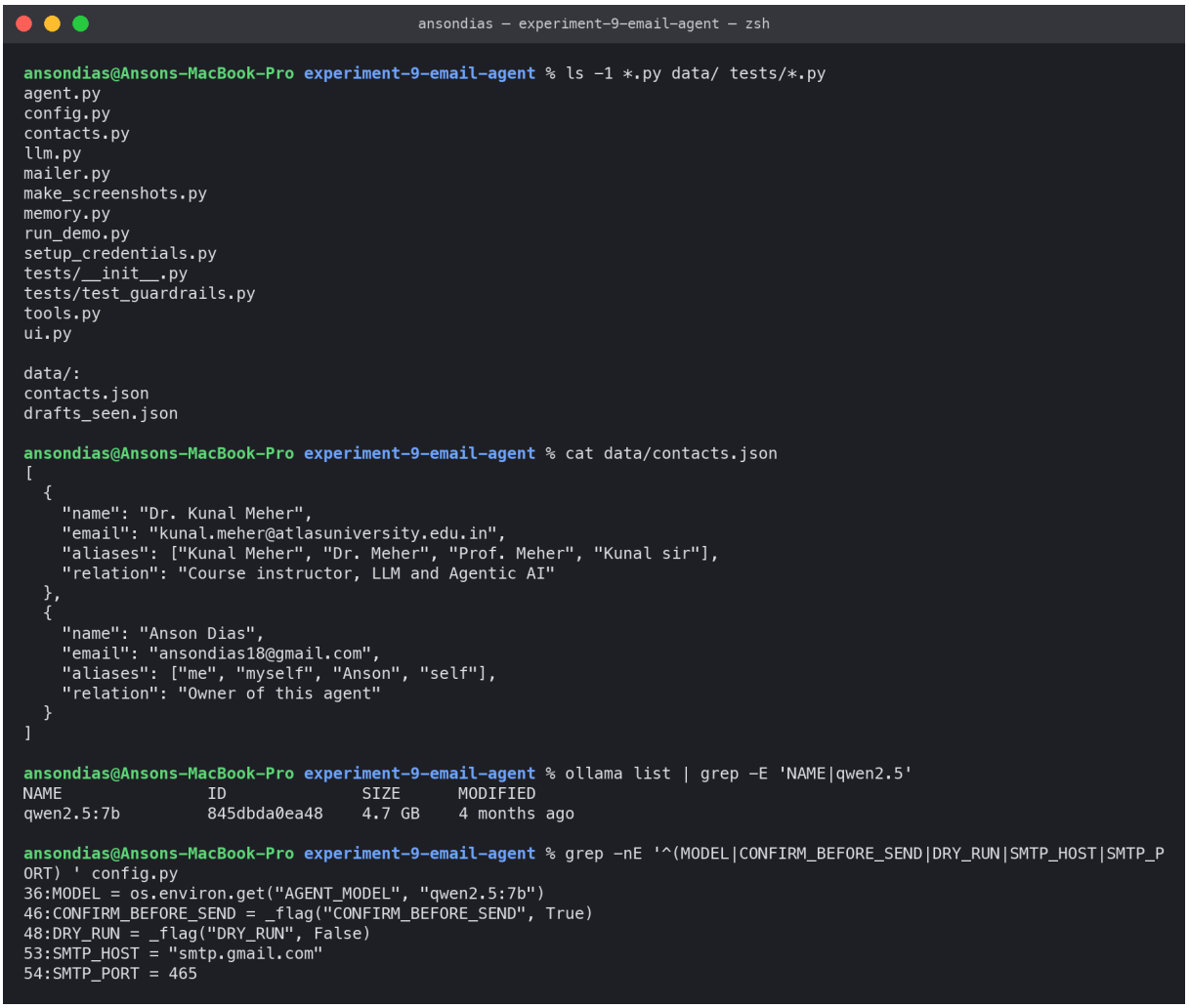


Fig. 1 — Project modules, contacts store, local LLM and the safety-relevant configuration.

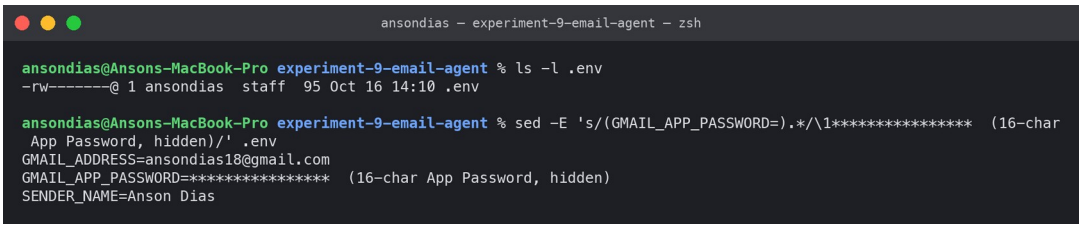


Fig. 2 — Gmail credentials stored in a private .env file (App Password masked).

Step 2 — Guardrail unit tests

Nine unit tests check the guardrails without the LLM or the network. They cover contact lookup with titles, aliases and typos; blocking a hallucinated address; accepting an address the user typed; refusing to save an invented address; no send and no retry after “n”; no duplicate send; memory persistence across restarts; and resend reusing memory with an adapted greeting. One test replaces the LLM with a scripted fake that deliberately guesses john@gmail.com. The guardrail blocks it, the confirmation step is never reached and nothing is sent. All 9 tests pass.

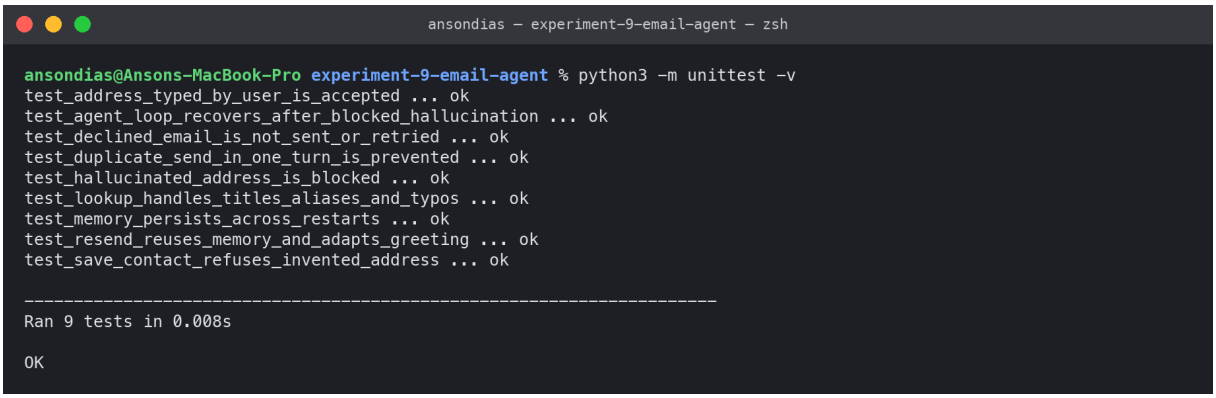


Fig. 3 — All nine guardrail tests pass.

Step 3 — Natural-language instruction → email to Dr. Meher (live send)

The trace in Fig. 4 covers every required stage of the problem statement:

- **1. Understand:** the schema-constrained call extracts intent=send_email, recipients=['Dr. Meher'], tone=formal and the key points.
- **2. Look up missing info:** the agent checks long-term memory (empty) and calls lookup_contact("Dr. Meher"), which resolves the alias to Dr. Kunal Meher <kunal.meher@atlasuniversity.edu.in>. The address is not generated by the LLM.
- **3. Draft:** the LLM writes the subject and an 88-word formal body with the requested points and the signature.
- **4. Confirm:** the guardrail confirms the recipient came from the contact store, shows the complete draft and waits. The answer was “y”.
- **5. Send:** smtplib delivered it through smtp.gmail.com:465 using the App Password, and Gmail accepted it (Message-ID shown).
- **6. Remember:** the email is stored as #1 in memory.json.

The whole instruction took 69.7 s with 5 LLM calls and 3 tool calls on the M1.

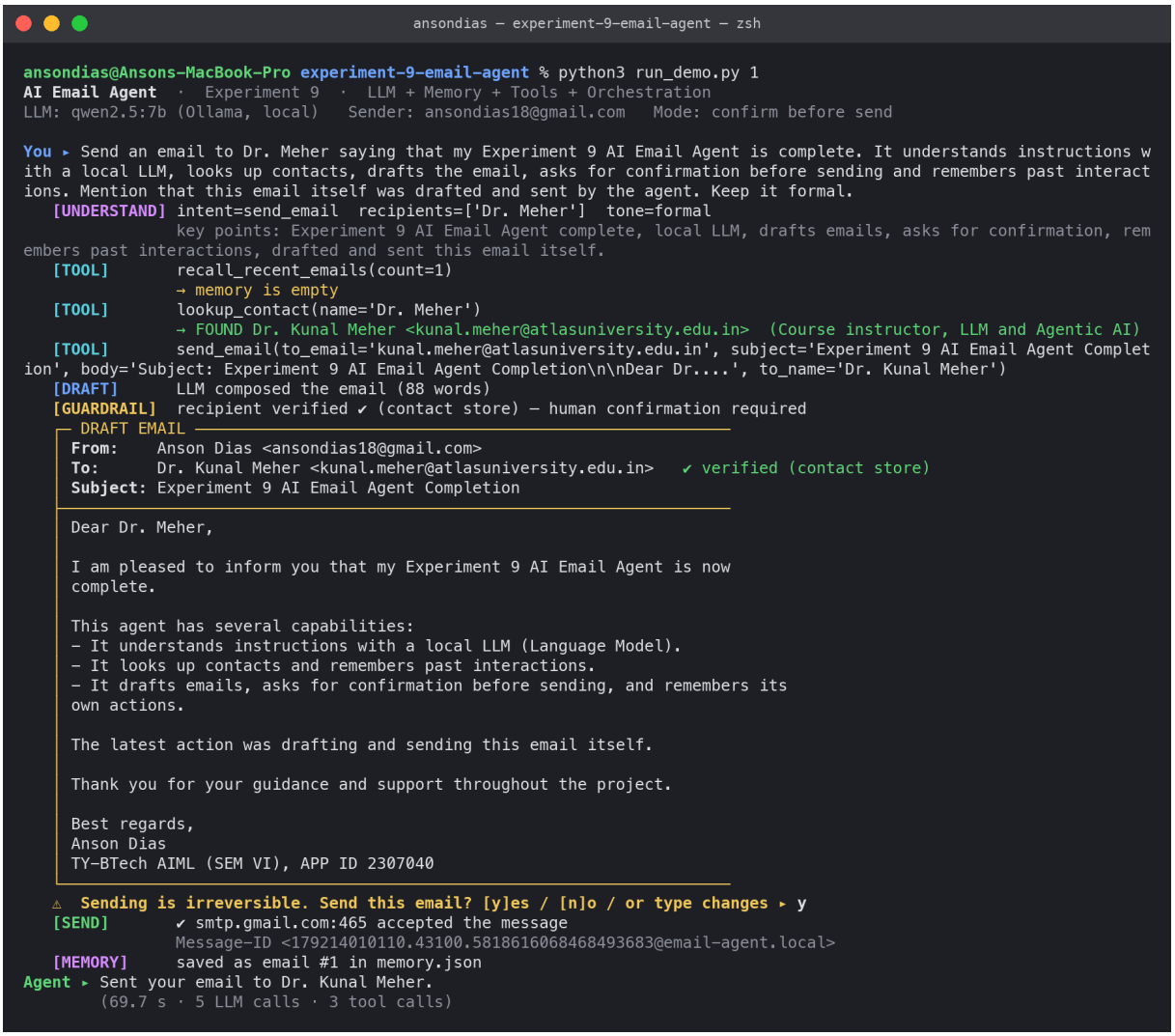


Fig. 4 — Instruction → understand → contact lookup → draft → human confirmation → Gmail SMTP send → memory.

Step 4 — Memory: “Send the same email to me too” (new process)

This is run in a fresh process, so the only link to Step 3 is the memory file. The agent recalls email #1 from long-term memory, resolves “me” to Anson Dias through the contacts store, and calls resend_email(email_id=1). The subject and body are reused word-for-word, and only the greeting changes to “Dear Anson,”. After confirmation the copy was sent (36.4 s, 3 LLM calls). One imperfection: the understanding step listed the earlier recipient as well (recipients=['Dr. Meher','yourself']). The planner still acted correctly because the “same email” rule and the memory tool define what is resent.

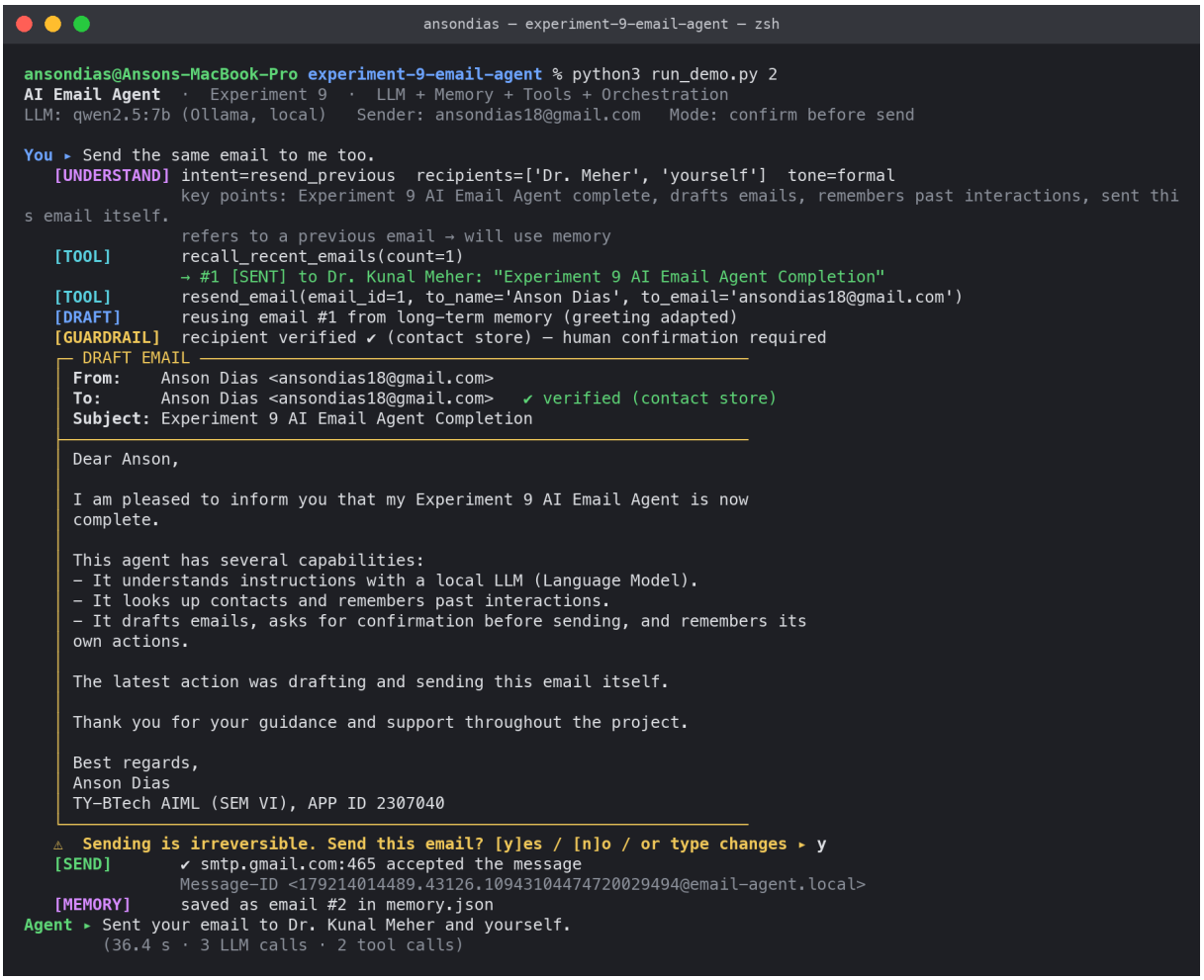


Fig. 5 — Follow-up handled from long-term memory: the stored email is reused exactly and sent to the owner.

Step 5 — Missing contact, hallucination blocked, user declines

Asked to “Email John a reminder about tomorrow’s 10 AM project meeting”, the 7B model skipped the lookup and **invented the address john@example.com** in its send_email call. This is exactly the failure the constraint warns about. The guardrail blocked it in code (“not in contacts and never given by the user”), the model received BLOCKED as its observation, and it asked the user for John’s address instead. When the user replied “It’s john.doe@example.com”, the short-term memory linked the reply to the pending task. The agent saved the contact (now verified, because the user typed it) and drafted the reminder with “tomorrow” turned into the real date, 17 October 2026. At the confirmation step the user answered “n”. The email was **not sent**, the draft was stored as DECLINED, and the agent did not retry.

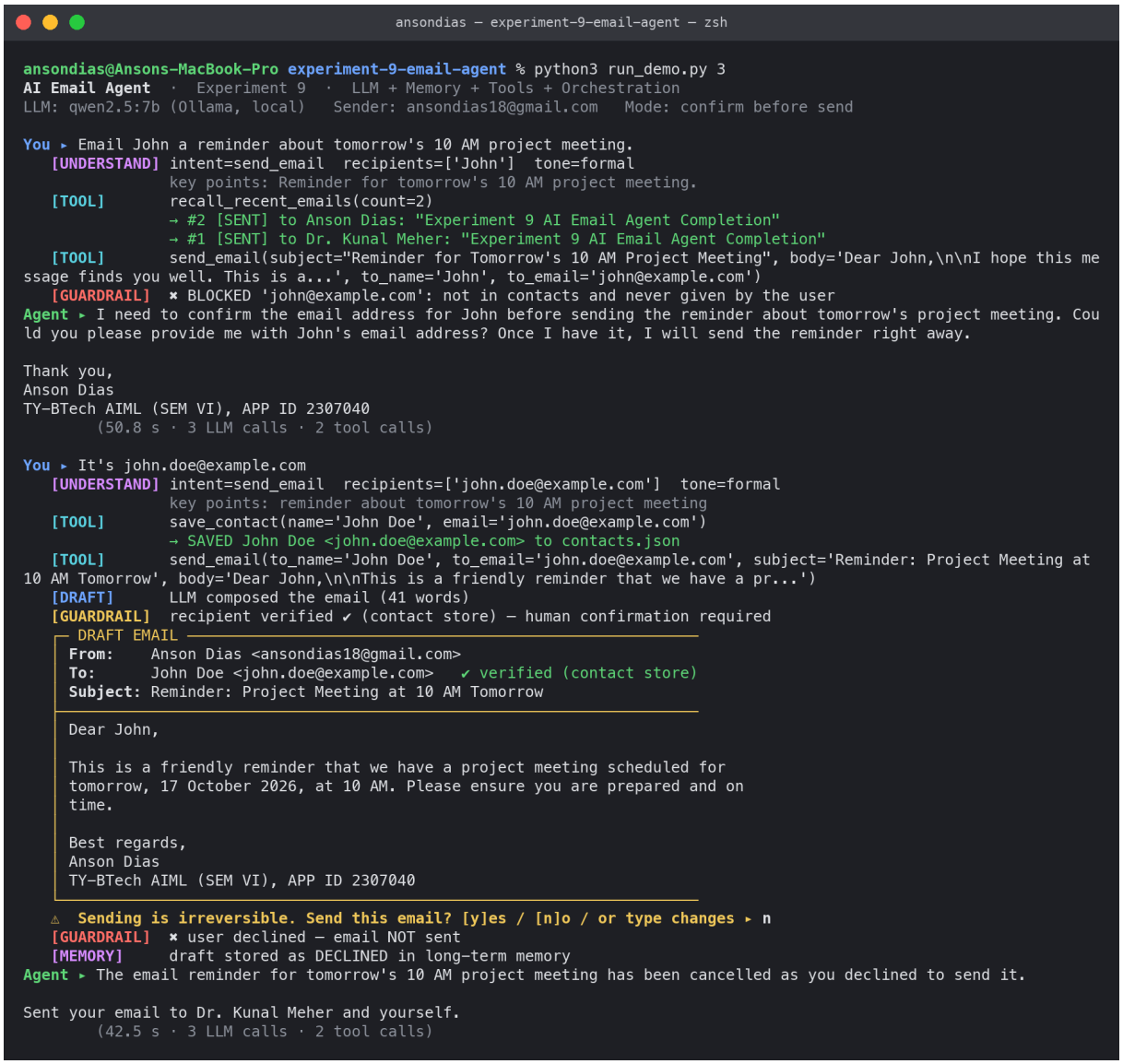


Fig. 6 — A hallucinated address is blocked; the agent asks, saves the user-given address, drafts, and respects the user's "no".

Step 6 — Long-term memory query

A new process is asked "What emails have I sent so far?". The understanding step classifies this as query_history, the agent calls recall_recent_emails(5) and answers from the stored records with their status: two SENT and one DECLINED. Fig. 8 shows the persisted state: the email log with status and time, the conversation turns used as short-term memory, and the contacts store, which now includes John Doe (added only because the user typed the address).

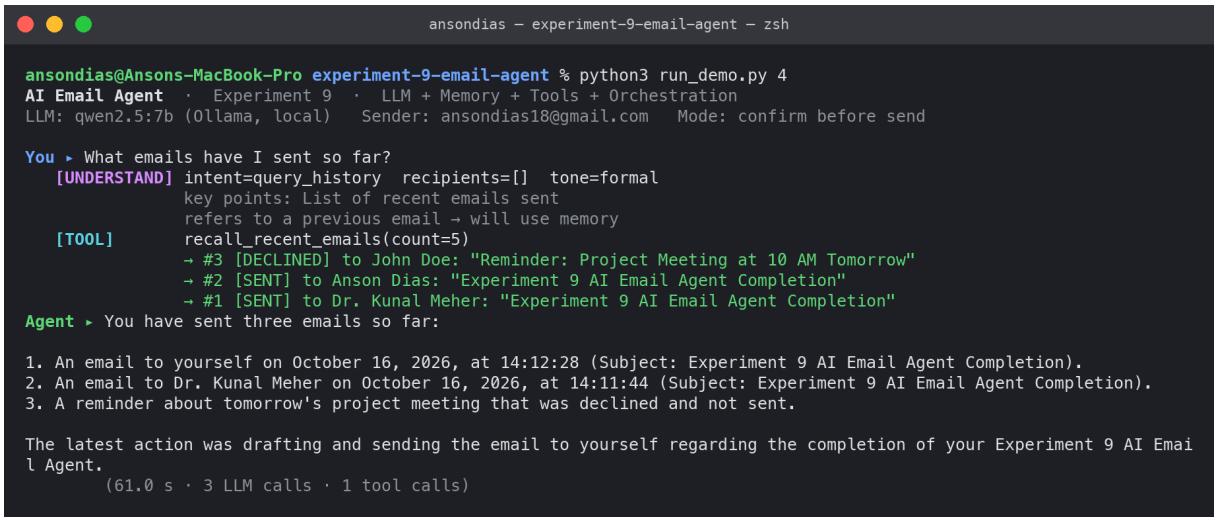


Fig. 7 — Answering a question about past interactions from long-term memory.

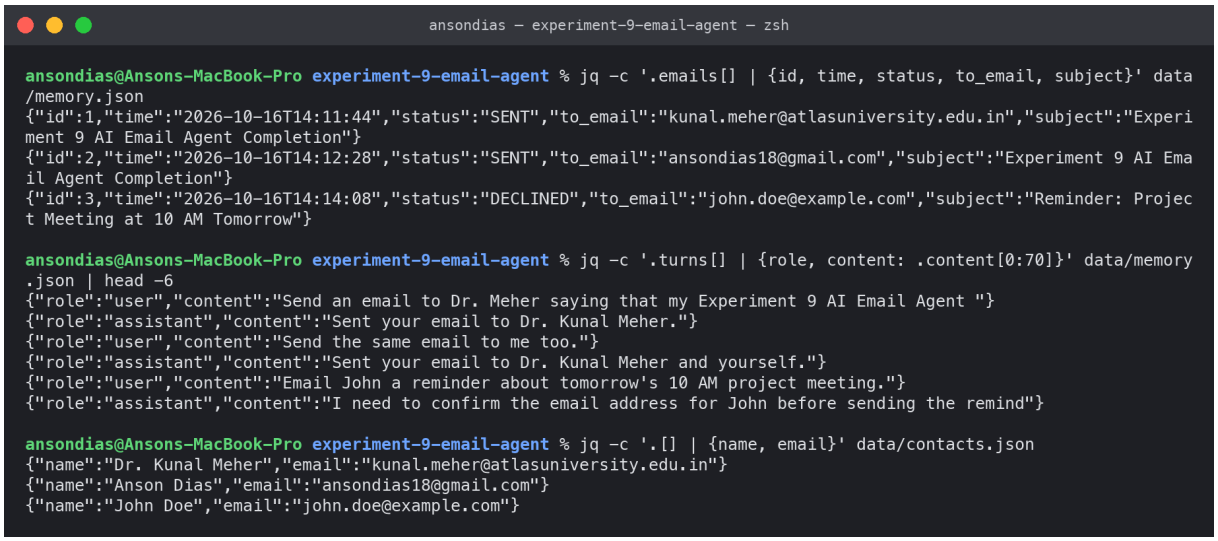


Fig. 8 — Persisted memory (emails + turns) and the updated contacts store after the session.

Step 7 — Independent delivery check (IMAP)

To confirm that SENT really means delivered, verify_delivery.py logged in to Gmail over IMAP with the same App Password and searched for each Message-ID. The email to kunal.meher@atlasuniversity.edu.in is in **[Gmail]/Sent Mail** (sent Fri, 16 Oct 2026 14:11:41 +0530). The copy to the owner is in both Sent Mail and the **Inbox** (14:12:24 +0530), so it made the round trip through Gmail. As required, the agent's email was delivered to kunal.meher@atlasuniversity.edu.in.

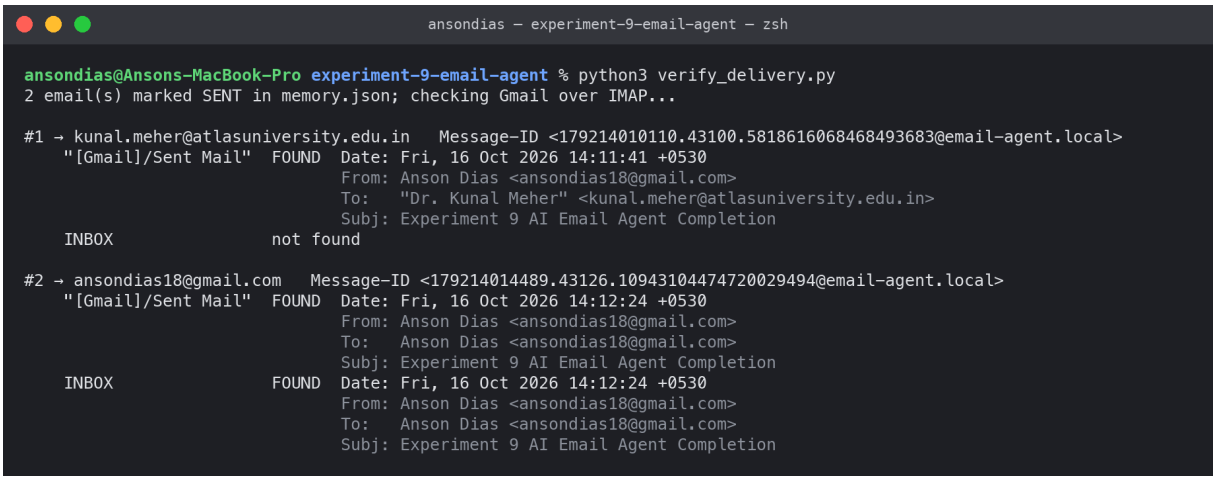


Fig. 9 — Gmail confirms both emails by Message-ID (Sent Mail, and the owner's Inbox for the copy).

Requirements and evidence

Requirement / constraint	Evidence
Understand intent and extract entities	[UNDERSTAND] lines in Figs. 4–7 (JSON-schema output)
Look up missing info (recipient email)	lookup_contact FOUND for Dr. Meher and "me"; NOT FOUND for John leads to asking the user (Figs. 4–6)
Draft the email	88-word formal draft (Fig. 4); dated reminder (Fig. 6)
Confirmation before an irreversible send	Full draft + y/n prompt each time; "n" means NOT sent (Fig. 6); unit tests (Fig. 3)
Send via Gmail SMTP with an App Password	smtp.gmail.com:465 accepted (Figs. 4–5); IMAP confirmation (Fig. 9)
Remember past interactions	"Send the same email to me too" in a new process (Fig. 5); history query (Fig. 7); memory.json (Fig. 8)
No hallucinated email addresses	Live: john@example.com BLOCKED (Fig. 6); unit tests for send and save_contact (Fig. 3)
Email received by kunal.meher@atlasuniversity.edu.in	Sent 16/10/2026 14:11 IST, Message-ID <179214010110.43100….@email-agent.local> (Figs. 4, 9)

Observations

- **Code-level guardrails are necessary, not optional.** The 7B model followed the "always look up" rule in Steps 3–4 but broke it in Step 5. A prompt alone would have let a guessed address through. The tool-boundary check caught it and turned the mistake into a question for the user.
- **Practical issues found while building the agent:** (i) Ollama sometimes dropped a tool call when the model put a long email body into the arguments and the JSON broke. Resending is therefore a separate resend_email(email_id) tool that copies the stored text instead of regenerating it. (ii) The model occasionally returned an empty reply after a tool result, so the orchestrator now sends a "continue" prompt. (iii) The model sometimes repeated "Subject:" inside the body, which a sanitiser removes. (iv) The extraction prompt once copied an example address, so examples with addresses were removed from it.
- **Small-model limitations remain cosmetic:** some chat replies carry over earlier text or add a signature (Step 5), and the extractor can over-list recipients (Step 4). Neither affects what is sent, because only the guarded tools can send and every email is shown in full first.

- **Latency:** 36–70 s per instruction (3–5 LLM calls) for a 7B model decoding at about 11 tokens/s on an 8 GB M1. That is acceptable for an assistant that waits for human confirmation anyway. A larger hosted model would be faster and need fewer orchestration fixes.

Conclusion:

An autonomous AI email agent was built and run end-to-end, combining the four parts of an agent. A local Qwen2.5-7B model on Ollama does the reasoning and planning. A ReAct orchestration loop runs schema-constrained understanding and native tool calling. Tools look up and save contacts, recall memory and send email. Memory has two layers: a short-term conversation window and a long-term email log persisted to disk. From plain-English instructions the agent extracted intent and entities, resolved recipients from a contacts store, drafted formal emails, asked for confirmation and delivered them through Gmail SMTP with a 16-character App Password. The email to kunal.meher@atlasuniversity.edu.in was sent and confirmed in Gmail. Follow-ups such as “send the same email to me too” and “what have I sent?” worked across restarts. The design constraints were enforced in code: a hallucinated address was blocked live, declined emails were never sent, and duplicates were prevented. The main lesson is that the LLM supplies the flexibility, while the orchestration and tool guardrails make it safe to let the agent take irreversible actions.